

Instituto Politécnico Nacional

Escuela Superior de Cómputo / UPIITA

Ingeniería en Inteligencia Artificial (Plan 2020)

Unidad de Aprendizaje: Bases de Datos

Práctica 1: Modelo Entidad Relación

Alumno: Cesar Javier Martinez Ruiz

Boleta: 2024630871 | **Grupo:** 3BV1

Ejercicio 1. Control de Versiones con Git y GitHub (Parte A: Investigación)

1. ¿Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo?

Un Sistema de Control de Versiones (VCS, por sus siglas en inglés *Version Control System*) es una infraestructura de software diseñada para registrar, auditar y gestionar las modificaciones realizadas sobre un conjunto de archivos a lo largo del tiempo. En esencia, actúa como un libro de contabilidad inmutable que almacena instantáneas (*snapshots*) del estado de un proyecto, permitiendo a los desarrolladores consultar el historial completo de cambios, identificar quién modificó cada línea de código, revertir el sistema a estados estables previos y bifurcar líneas de desarrollo independientes.

En un entorno de trabajo colaborativo, un VCS resuelve el problema crítico de la **conurrencia destructiva** y la **asincronía de integración**. Sin un VCS, el desarrollo compartido depende de mecanismos rudimentarios y propensos al error humano (como compartir carpetas comprimidas por correo o servidores compartidos con sufijos tipo `archivo_final_v2_revisado.sql`), lo que irremediablemente causa sobreescrituras accidentales donde el trabajo de un ingeniero elimina el de otro. El VCS resuelve esto mediante algoritmos de control de concurrencia distribuida: cada colaborador trabaja en un clon local completo y, al integrar sus modificaciones, el sistema analiza matemáticamente las diferencias línea por línea mediante grafos acíclicos dirigidos (DAG), fusionando cambios automáticos no conflictivos y forzando la resolución consciente de divergencias antes de aceptar cualquier integración.

2. Diferencia entre Git y GitHub

Existe una confusión frecuente entre ambos términos, pero representan niveles tecnológicos fundamentalmente distintos:

Git es una **herramienta de software y un protocolo de control de versiones distribuido**, de código abierto, creado por Linus Torvalds en 2005. Funciona localmente en la máquina del usuario a través de la terminal o interfaz de comandos; no requiere conexión a Internet para operar, calcular diferencias, generar confirmaciones (commits*) o navegar por el historial. Git es el motor criptográfico subyacente que gestiona el árbol de objetos y el historial de cambios.

GitHub es una **plataforma comercial en la nube (SaaS - Software as a Service)**, propiedad de Microsoft, diseñada para alojar repositorios de Git remotos y ofrecer servicios colaborativos sobre ellos. GitHub añade una capa de interfaz gráfica web, control de acceso basado en roles, seguimiento de problemas (issues), flujos de integración y despliegue continuo (GitHub Actions), discusiones, revisiones de código mediante Pull Requests* y herramientas de gestión de proyectos. En términos simples: Git es el motor del automóvil, mientras que GitHub es la autopista y la infraestructura de servicios que permite a muchos vehículos circular y coordinarse de forma masiva.

3. Glosario Conceptual y Técnico con Ejemplos Prácticos

Ejemplo: La carpeta `practical1-bd` inicializada con `git init`, que almacena desde el primer archivo SQL hasta los esquemas de bases de datos.

Ejemplo: Registrar la creación del esquema de tablas con el comando `git commit -m "feat(schema): definir tablas de usuarios y prestamos"`.

Ejemplo: Crear la rama `feature/entorno-docker` para configurar los contenedores sin comprometer la rama `main`.

Ejemplo: Ejecutar `git merge feature/entorno-docker` sobre `main` para integrar la infraestructura de Docker ya probada.

Ejemplo: Si dos desarrolladores modificaron concurrentemente la línea 12 de `compose.yaml` asignando puertos distintos (uno `5432:5432` y otro `5433:5432`), Git suspende el merge y marca el archivo con delimitadores `<<<<<<<`, `=====` y `>>>>>>>` para que el desarrollador elija la resolución correcta.

* **Pull Request (PR):** Solicitud formal en una plataforma como GitHub para que los cambios desarrollados en una rama secundaria sean revisados, discutidos y evaluados por el equipo antes de ser fusionados a la rama principal.

Ejemplo: Un PR titulado "*feat: incorporar modelo relacional y DDL*" donde el docente o compañero revisa las cardinalidades antes de aceptar la integración.

* **Archivo `.gitignore`:** Archivo de configuración en texto plano ubicado en la raíz del repositorio que enumera patrones de archivos, carpetas o extensiones que Git debe omitir intencionalmente del seguimiento de versiones.

Ejemplo: Ignorar contraseñas y volúmenes de Docker mediante las líneas `.env` y `pgdata/` para no exponer credenciales ni saturar el repositorio con binarios temporales.

* **Archivo `README.md`:** Documento principal escrito en formato Markdown que funge como la portada y manual de bienvenida de un proyecto. Informa qué hace el sistema, quién es el autor, cómo instalarlo y cómo navegar por sus componentes.

Ejemplo: El archivo `README.md` de esta práctica con los datos del alumno Cesar Javier Martinez Ruiz, boleta 2024630871 y el índice a los reportes técnicos.

4. Flujo de trabajo basado en ramas y valor de la revisión por pares (*Peer Code Review*)

Un flujo de trabajo basado en ramas (como *GitHub Flow* o *GitFlow*) establece que la rama principal (`main` o `master`) representa en todo momento un estado estable, verificado y listo para producción. Cualquier trabajo nuevo —sea una nueva característica (*feature*), corrección de error (*bugfix*) o refactorización— debe originarse en una rama secundaria aislada.

Este flujo exige que la integración hacia `main` se realice exclusivamente a través de un *Pull Request* sujeto a **revisión por pares (*Peer Code Review*)**. La revisión de código es una práctica de ingeniería fundamental porque:

1. **Detección temprana de anomalías:** La probabilidad de que un desarrollador pase por alto vulnerabilidades de seguridad, cuellos de botella o violaciones a las formas normales de bases de datos se reduce significativamente cuando otro par analiza el diseño con perspectiva fresca.

2. **Homogeneidad arquitectónica y de estilos:** Garantiza que las convenciones de código, nombrado de entidades y documentación se mantengan consistentes en toda la base de código.

3. **Distribución horizontal del conocimiento:** Evita los "silos de información" donde solo un miembro del equipo comprende un módulo crítico. Al revisar el trabajo ajeno, todo el equipo se familiariza con el funcionamiento del sistema, elevando la resiliencia técnica del colectivo.

Ejercicio 2. El Sistema Gestor en un Contenedor: Docker (Parte A: Investigación)

1. ¿Qué es un contenedor y en qué se diferencia de una máquina virtual?

Un **contenedor** es una unidad estándar de software que empaqueta el código de una aplicación junto con todas sus bibliotecas, dependencias y configuraciones de entorno necesarias para ejecutarse de manera confiable y aislada en cualquier infraestructura informática. A diferencia de las soluciones tradicionales, los contenedores operan mediante la virtualización a nivel de sistema operativo (*OS-level virtualization*), aprovechando características nativas del núcleo Linux como *namespaces* (aislamiento de procesos, red y sistemas de archivos) y *cgroups* (limitación de recursos de CPU, memoria y E/S).

• Criterio • Contenedor (Docker / Podman) • Máquina Virtual (KVM / VirtualBox / VMware)
• Tiempo de Arranque • Subsegundos a pocos segundos . Solo inicia los procesos de la aplicación en el espacio de usuario. • Minutos . Requiere el arranque completo de la BIOS virtual, el kernel del sistema operativo invitado y servicios de inicio.
• Tamaño y Huella • Ligero (Decenas a cientos de MB) . Comparte el kernel del sistema operativo anfitrión (<i>host</i>); no duplica capas del sistema base. • Pesado (Varios GB a decenas de GB) . Incluye una copia completa e independiente del sistema operativo invitado y sus binarios.
• Aislamiento • Aislamiento a nivel de procesos . Comparte el mismo kernel con el host. Un fallo crítico en el kernel puede comprometer al resto de contenedores. • Aislamiento estricto por hardware . Cada VM corre sobre un hipervisor Tipo 1 o 2 con memoria y CPU emuladas o asignadas por hardware.

2. Conceptos Fundamentales: Imagen, Contenedor, Volumen y Puerto Publicado

3. ¿Por qué el volumen es indispensable y qué ocurre exactamente si no se declara?

El volumen es indispensable debido a la naturaleza intrínseca del ciclo de vida de los contenedores, los cuales están diseñados arquitectónicamente para ser **efímeros y descartables** (*stateless by design*). Cuando un contenedor realiza operaciones de escritura (como inserciones de tuplas o creación de índices en PostgreSQL), estas modificaciones se almacenan en su capa de lectura y escritura (*Copy-on-Write storage layer*).

Si un desarrollador levanta un sistema gestor de bases de datos como PostgreSQL **sin declarar un volumen** y crea tablas o registros:

1. Mientras el contenedor continúe en ejecución o simplemente se pause (`docker stop`), los datos parecerán estar presentes.
2. Sin embargo, en el momento en que el contenedor se actualice, se destruya (`docker rm -f pg-practical`) o el host se reinicie y recree el contenedor, **la capa de lectura/escritura asociada al identificador del contenedor se destruye irreversiblemente**.
3. Como resultado, **toda la información, tablas, usuarios y esquemas creados desaparecen de manera definitiva**, pues nunca existió un enlace hacia el almacenamiento durable del disco duro del host.

El uso de un volumen con nombre (como `pgdata:/var/lib/postgresql/data`) o un *bind mount* desacopla el ciclo de vida del dato del ciclo de vida del contenedor: el contenedor puede ser destruido, recreado o actualizado a una nueva versión del motor (ej. PostgreSQL 16 a 17) manteniendo la integridad y persistencia absoluta de los datos almacenados en disco.